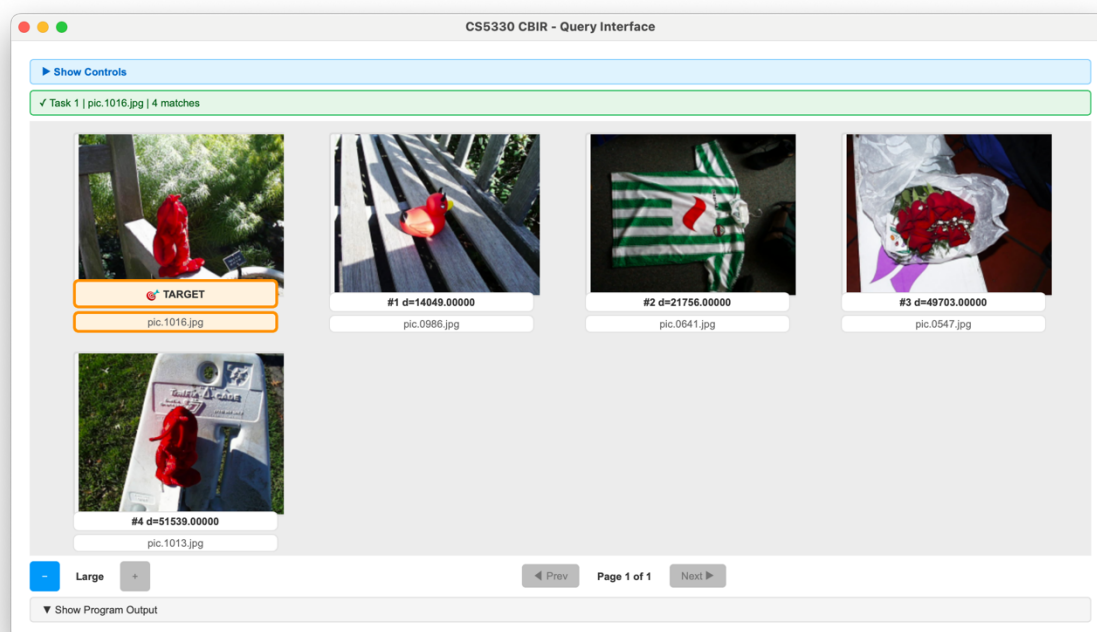


A short description of the overall project:

This project builds a Content-Based Image Retrieval system that finds images similar to a target image using different feature extraction methods. Six methods were implemented, including basic pixel matching, rg chromaticity histograms, multiple histograms, combined color-texture features, ResNet18 embeddings and a custom grass detection feature based on HSV color space.

A PyQt6 graphical interface is built as an extension, which makes it easier for users to see the retrieved matches and compare results among different tasks. Features such as CSV caching and zoom controls improve usability. The results show that deep learning embeddings perform better for semantic similarity, while classical features are effective for color and texture similarity.

Task 1:

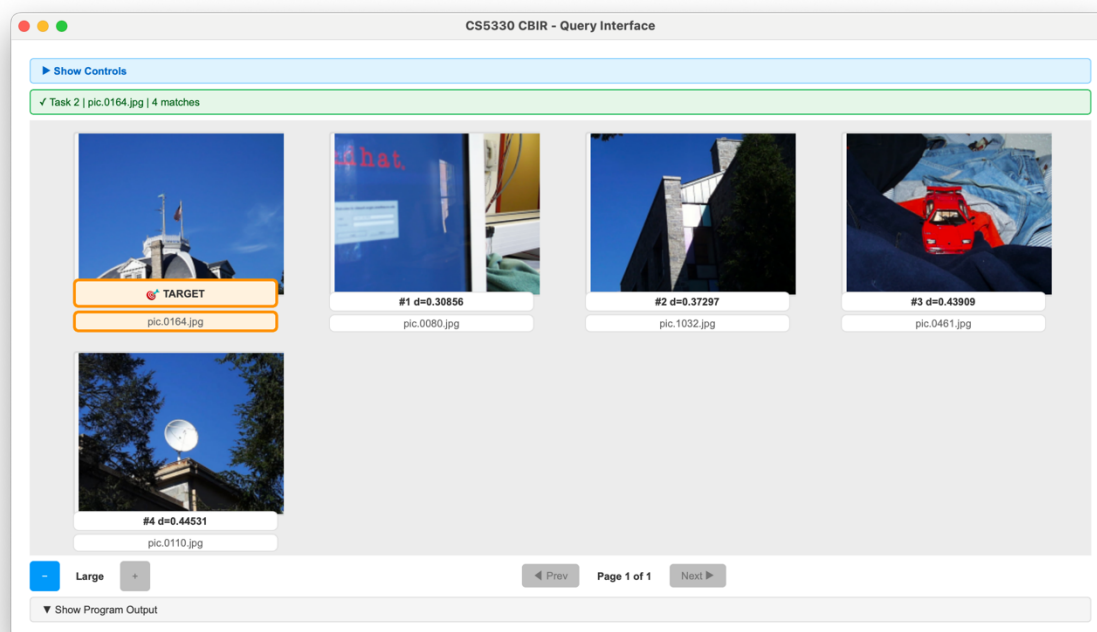


Description:

In task 1, we use the 7×7 square in the middle of the image as a feature vector. And we use sum-of-squared-difference as the distance metric. This works because images with similar centers will have similar feature vectors.

But based on the retrieval results, while the top matches all contain a red object, they are not all semantically related. This shows the limitation of relying solely on pixel differences.

Task 2 – Color Histogram Matching



Description:

In task 2, I used a global color histogram (rg chromaticity histogram) to represent each image.

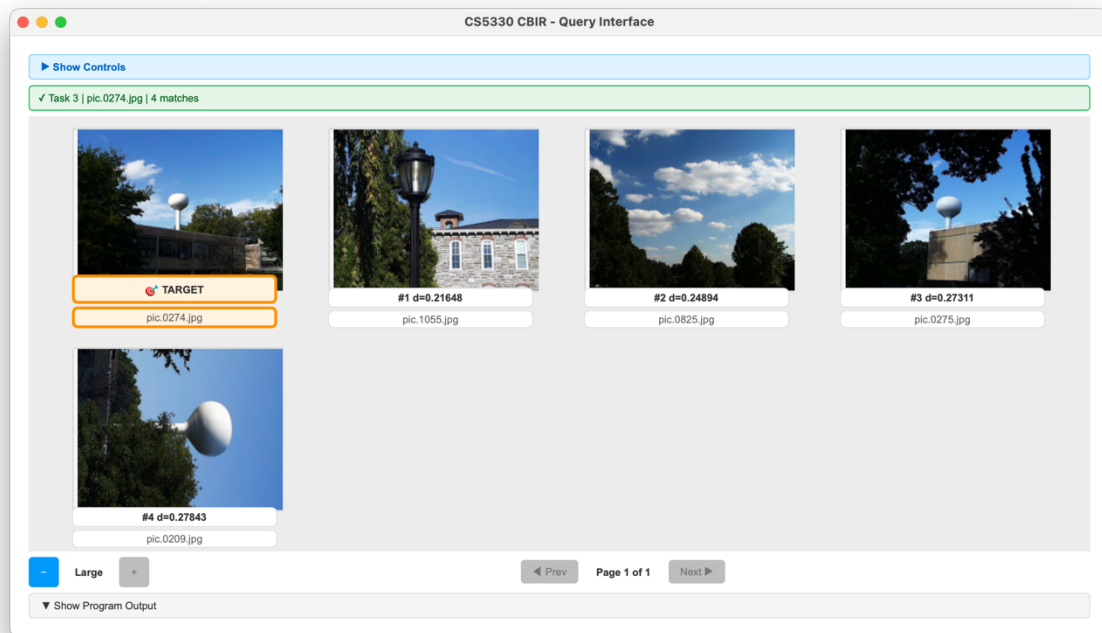
I assigned each pixel to a bin based on its color(r,g). And then normalized so that comparison is invariant to image size.

And I used histogram intersection distance to compare the images.

As shown in the figure, the top matches contain large regions of sky or similar blue chunks, which makes sense because the histogram is capturing overall color distribution.

But it's not capturing other characteristics like structure or semantics, so images with similar colors but different structures can still be retrieved.

Task 3 – Multi-Region Histogram



Description:

In task 3 I used the following two separate histograms:

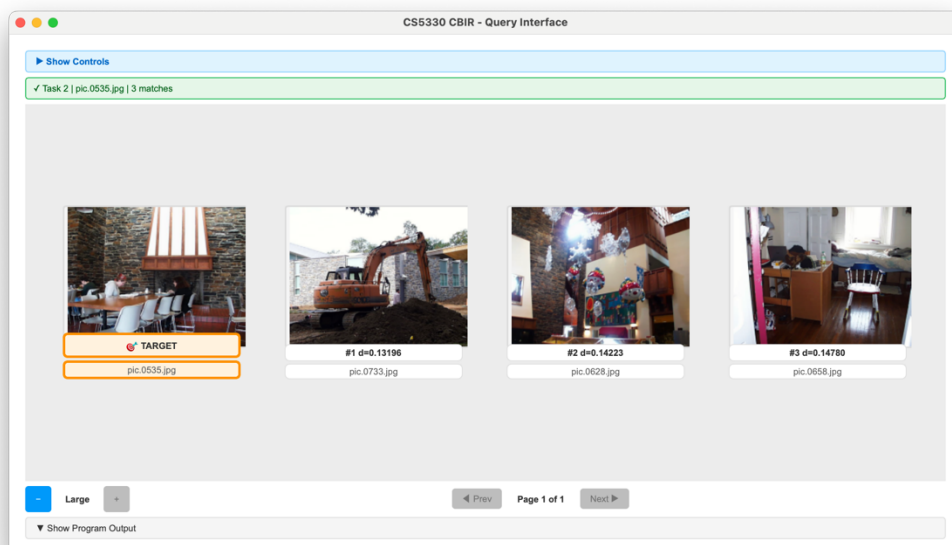
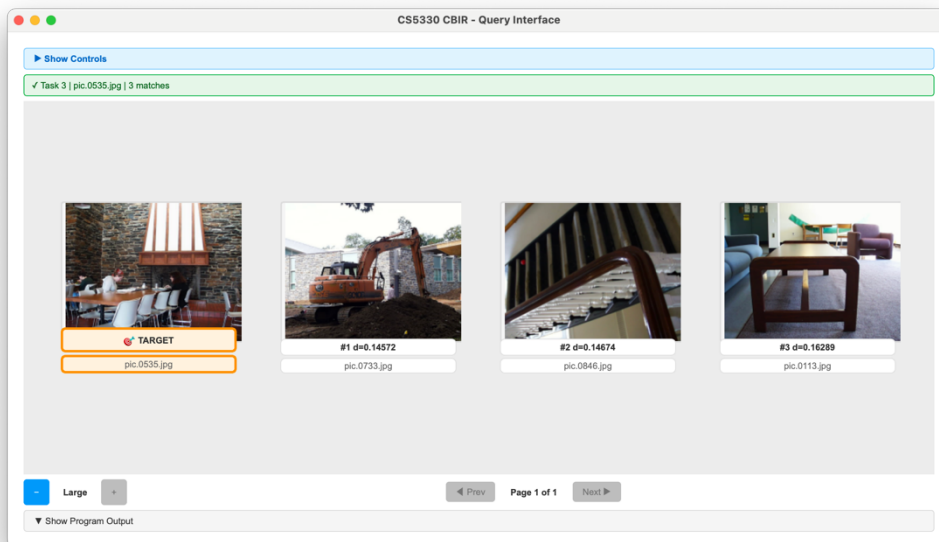
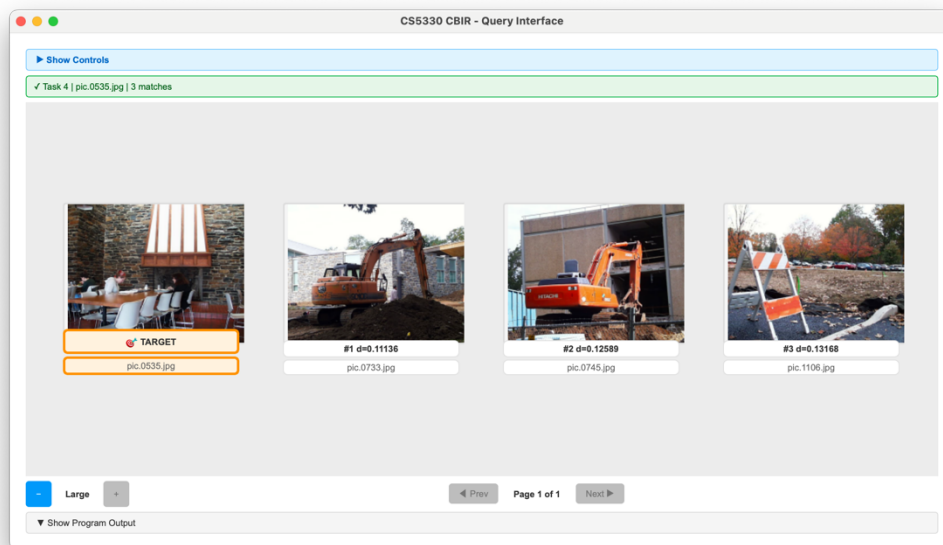
h_{whole} : entire image

h_{center} : center $50\% \times 50\%$ region

My reasoning:

Different spatial regions capture different information. Whole image can tell us overall color distribution, while center of image often contains the main subject.

It's notable that among the top 4 matches, there are 2 that contains the same object that appears in the target, which indicates using multiple histograms for different regions of the images is helpful.

Task 4 – Texture and Color:

Description:

In task 4, I used a rg chromaticity 2D hist with 16 bins for whole image(reused code from task 2) to capture the global color distribution. And my texture strategy is using both Magnitude and Orientation:

Magnitude: How strong are the edges?

Orientation: What direction are the edges?

So, the feature is concatenated like this:

[0:255] → Color histogram (rg, 16×16)

[256:271] → Magnitude histogram (16 bins)

[272:289] → Orientation histogram (18 bins)

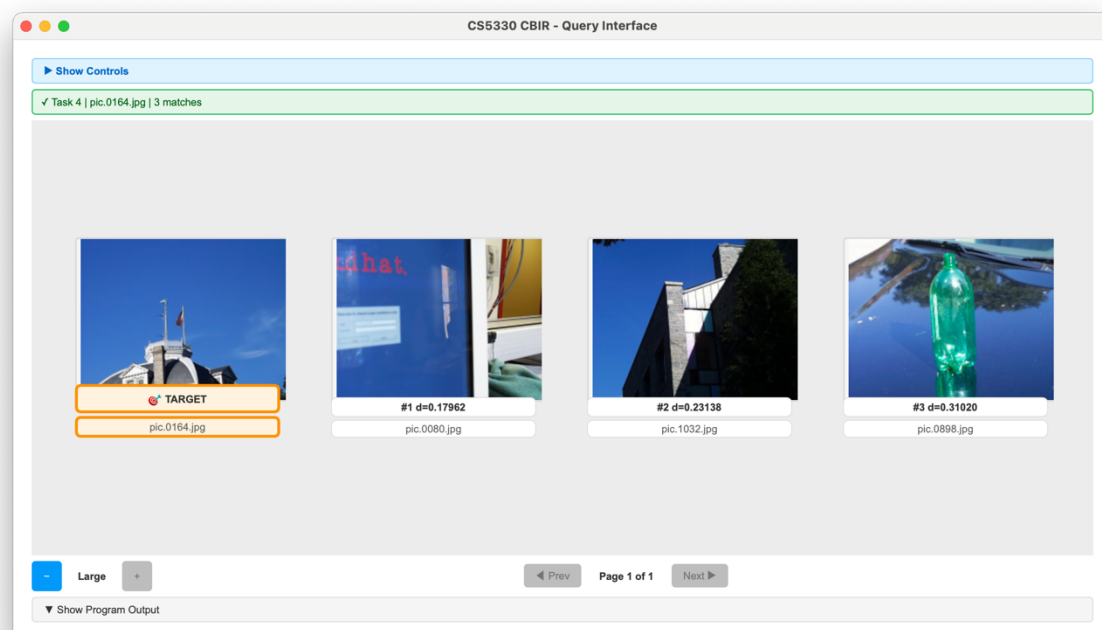
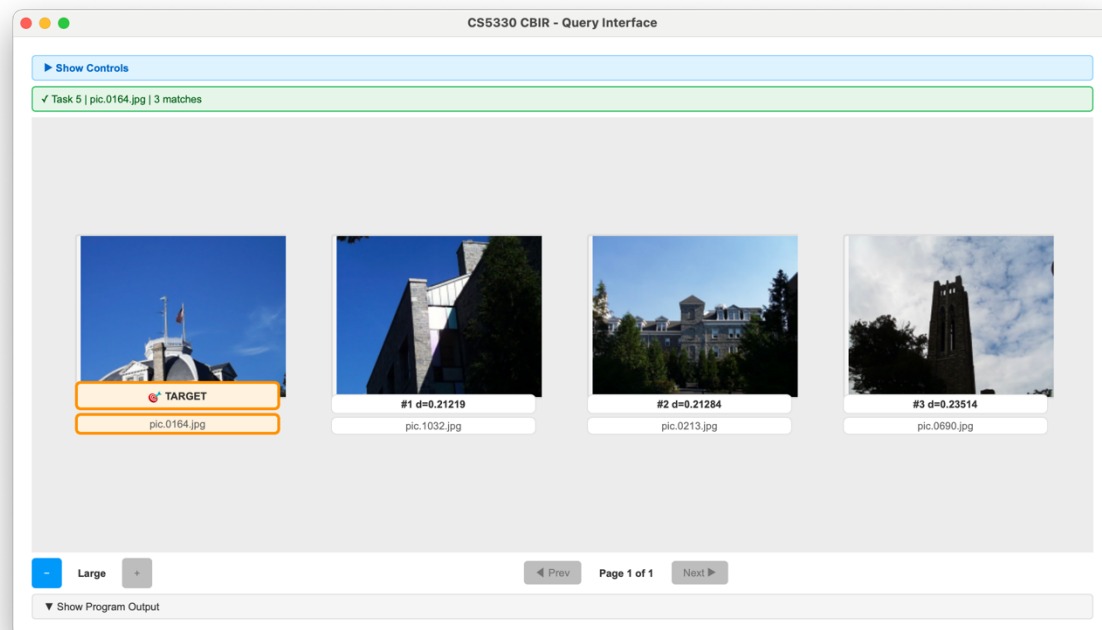
Compare the results:

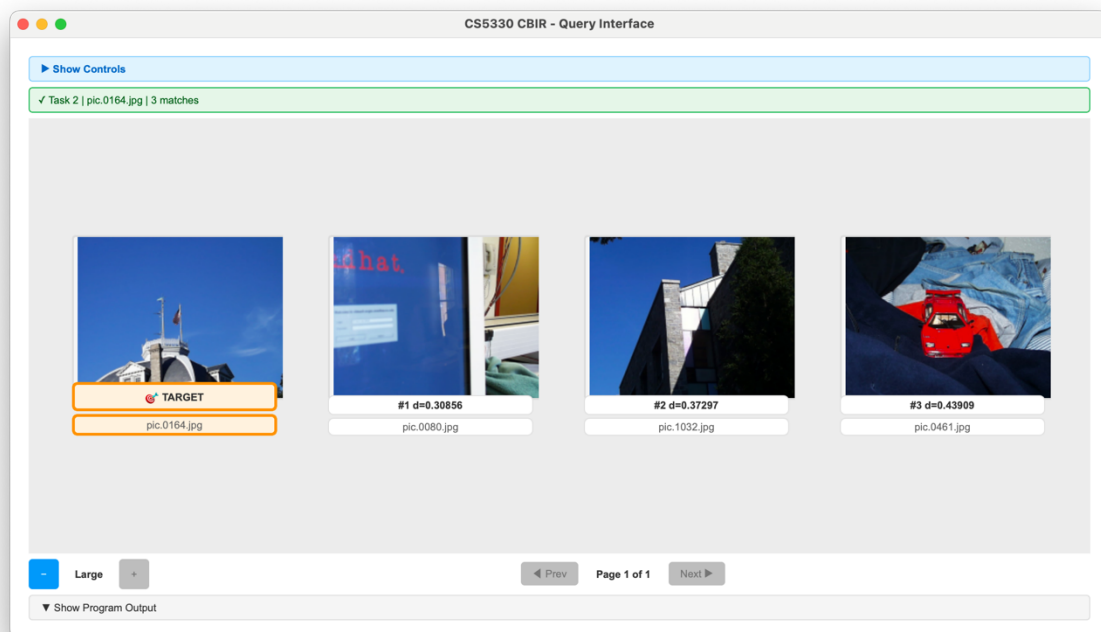
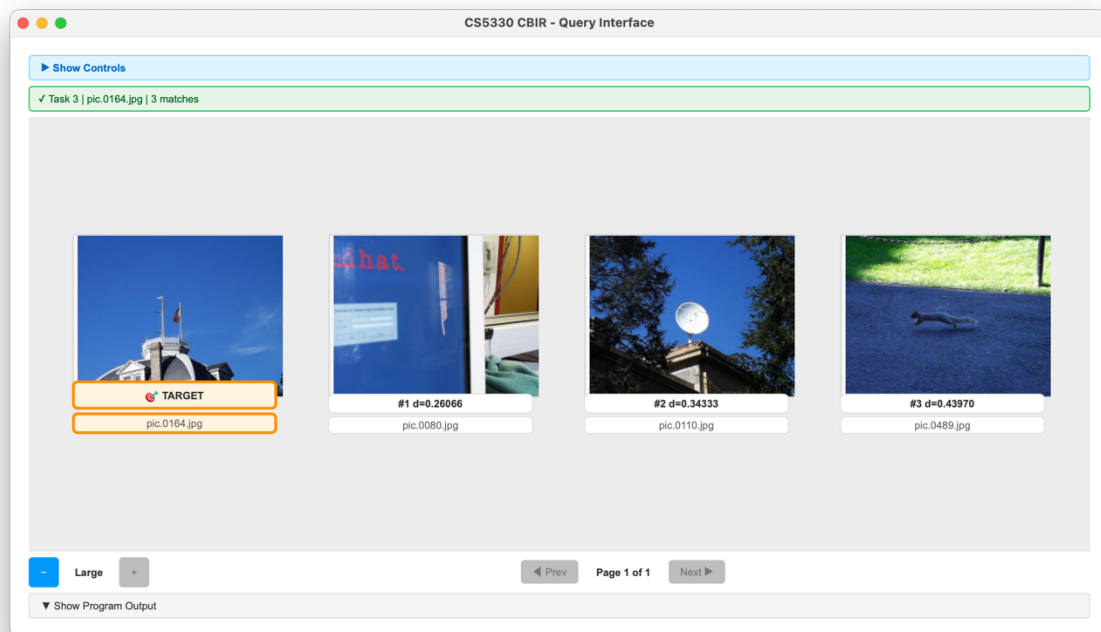
Results in task 2 only contain similar color chunks, not necessarily in the same region of the image.

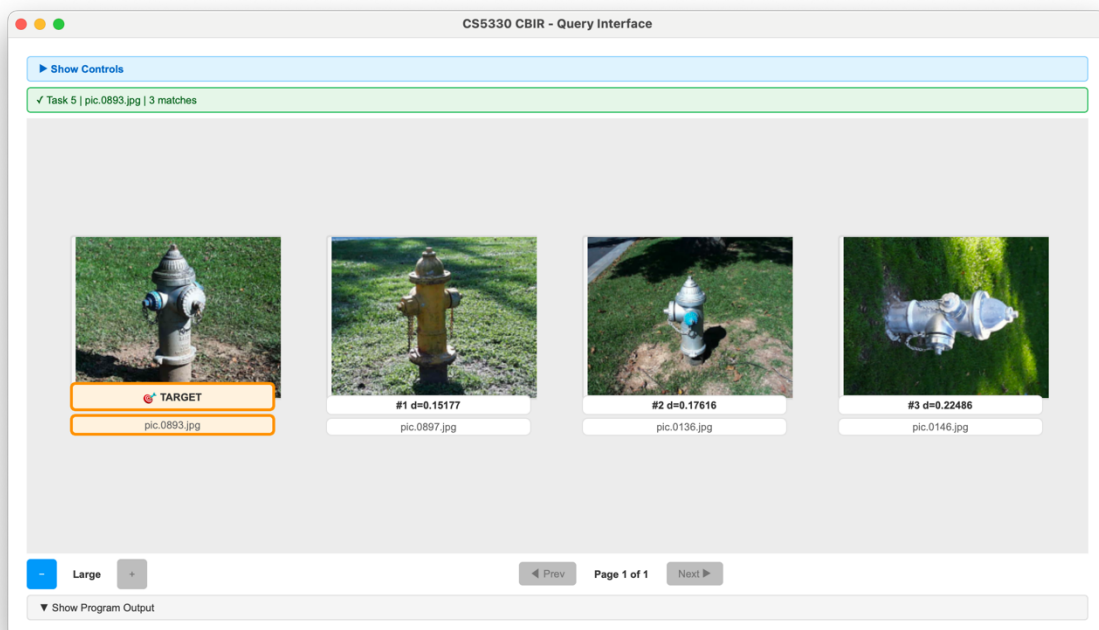
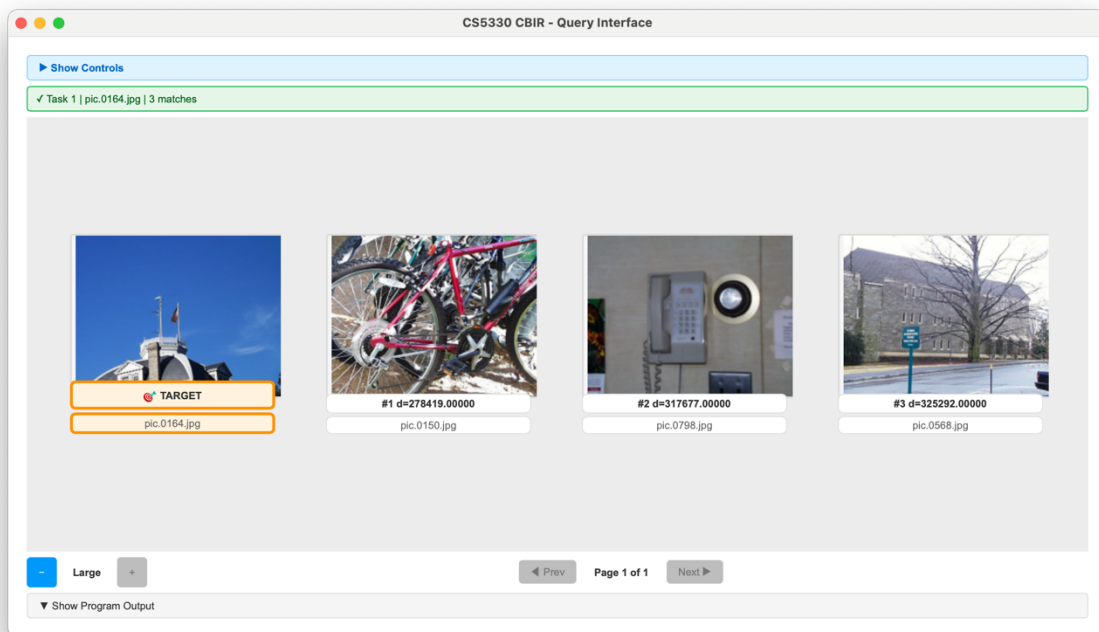
Results in task 3 contain similar color chunks, and an object of similar color always shows in the center of the image because we have a histogram for center area.

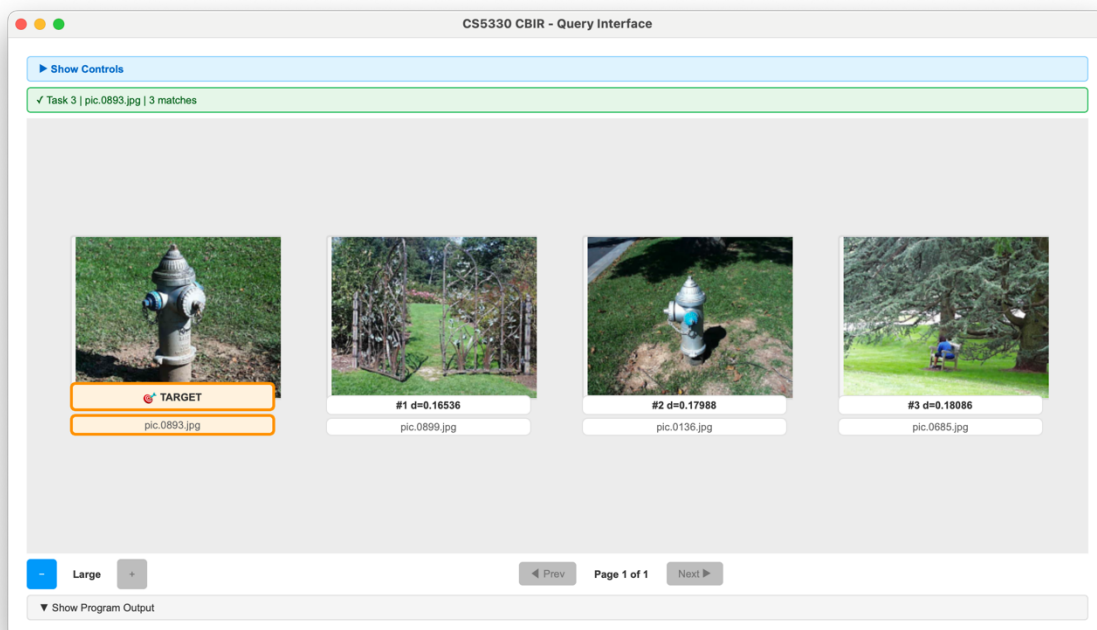
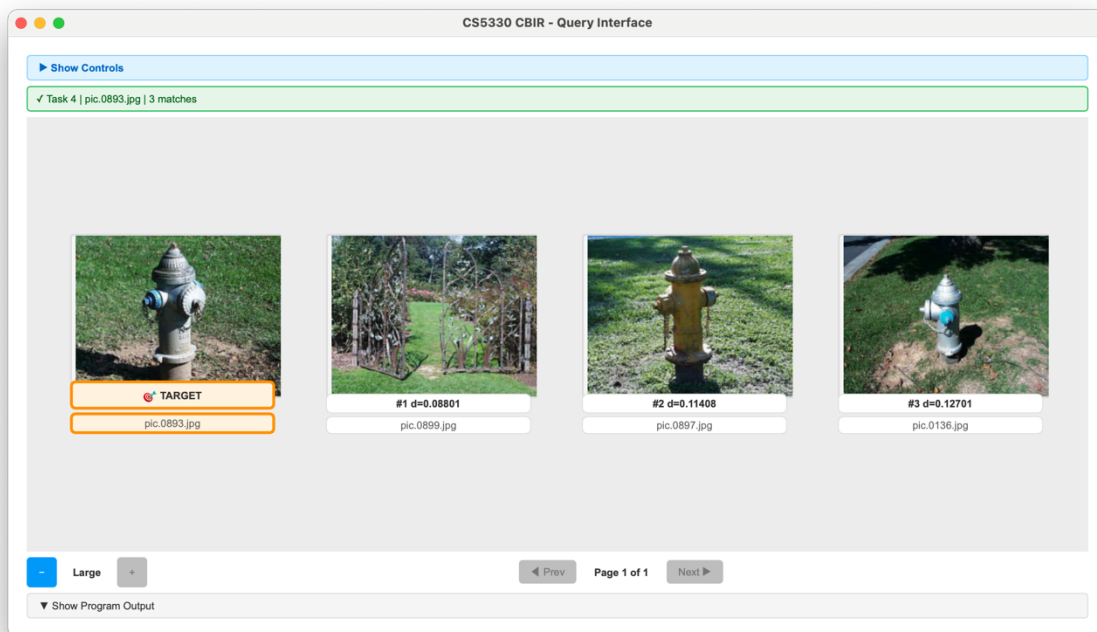
Results in task 4, the results all contain orange or brown items, and they are of similar shapes, like stripes, edges, which shows we have captured the structural characteristics and it does help in finding the right match and increasing the retrieval quality.

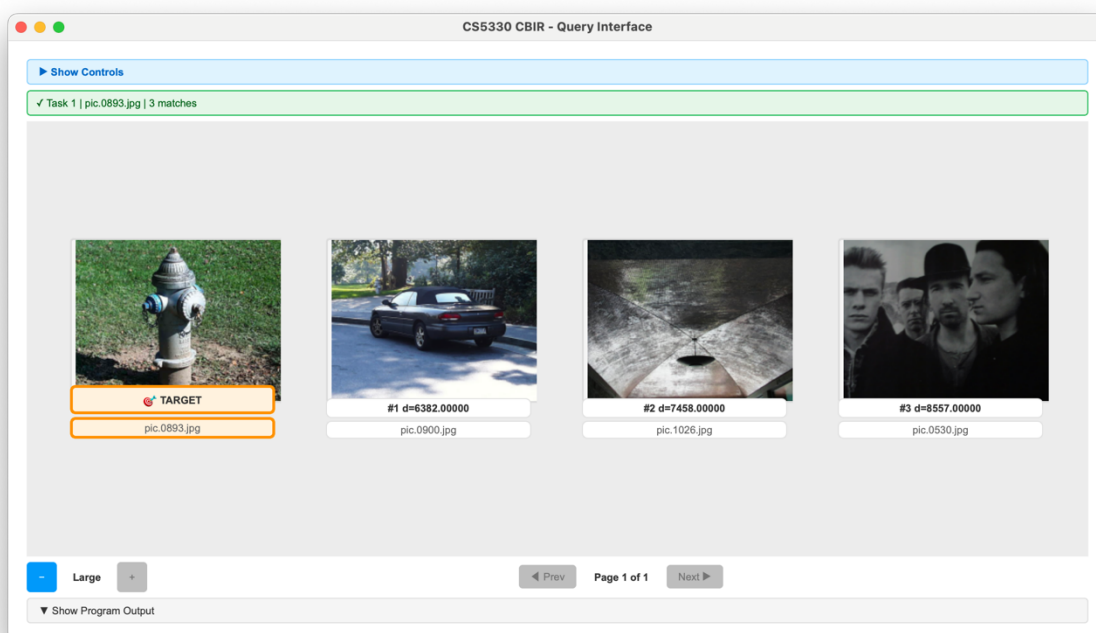
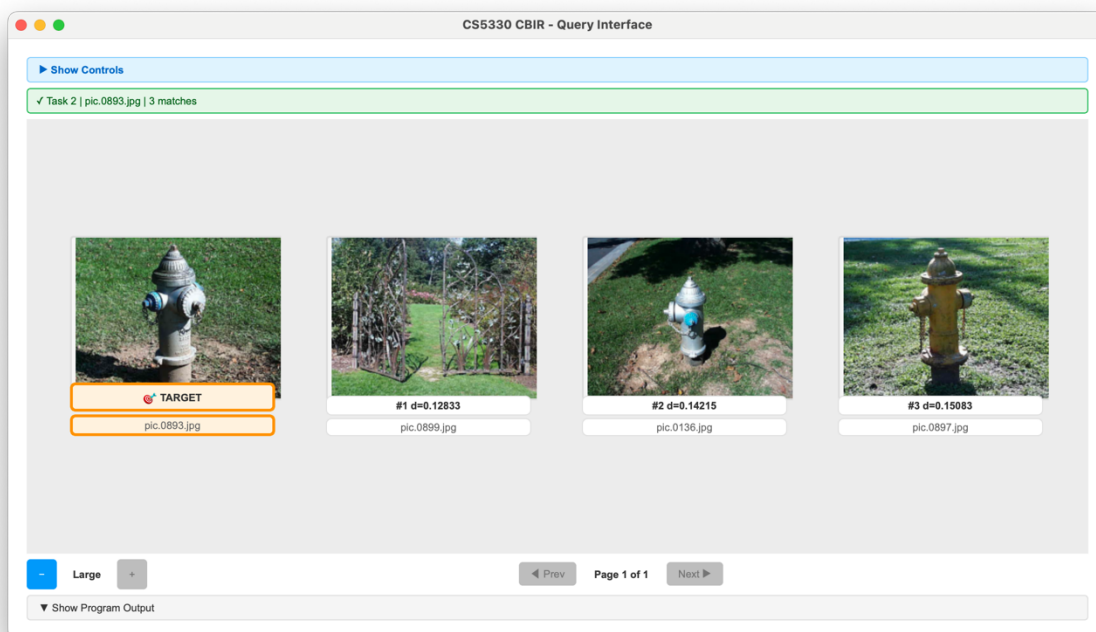
Task 5 – Results and Comparison











Description:

In Task 5, I used pre-trained deep network embeddings as feature vectors for image retrieval. Each image is represented by a 512-dimensional vector. Deep network embeddings capture high-level semantic information, so the retrieved results all have high similarity on object level instead of on color or texture level compared to previous tasks. For image pic.0893.jpg, what we get are all architectures. For image pic.0164.jpg, what we get are all fire hydrants.

I used Cosine distance because it performs well in high-dimensional spaces.

Task 6: Compare DNN Embeddings and Classic Features

I compared classic features (Tasks 2-4) with DNN embeddings (Task 5)

on three test images: pic.1072 (roses), pic.0948 (mascot), and pic.0734 (construction equipment).

Results:

For pic.1072 (roses), DNN successfully found all the flowers, even though not necessarily of the same color as target. Classical features mainly focus on capturing color or texture, so they found some photos of roses of same color, but also some other non-rose objects.

So, if the goal is to find flowers, DNN undoubtedly performs the best. But if the goal is to find a photo containing the similar color, DNN is not necessarily the winner.

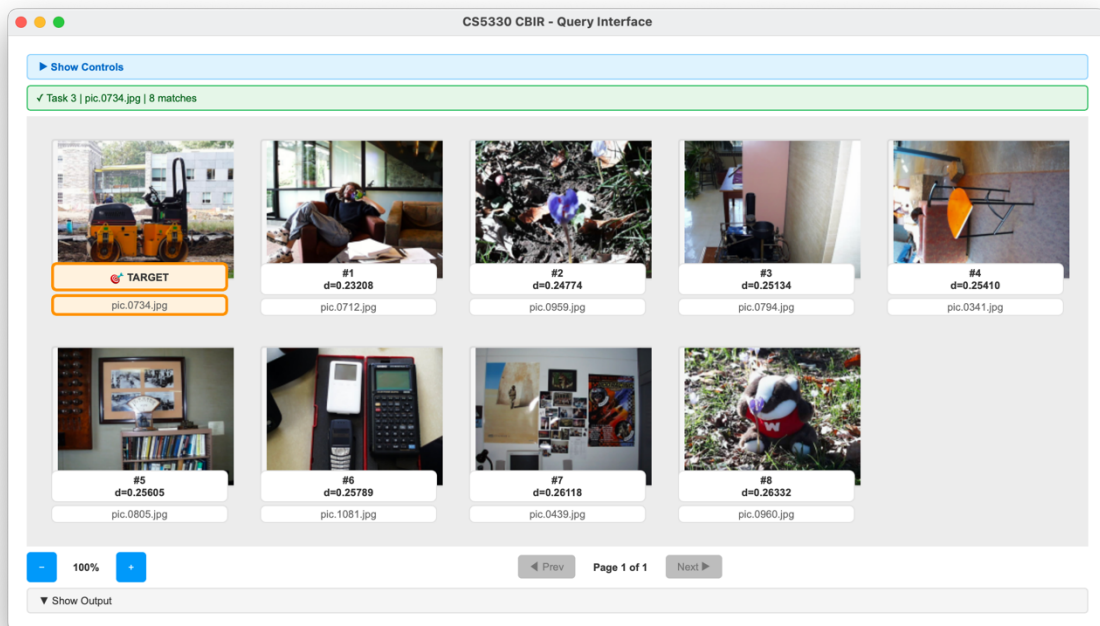
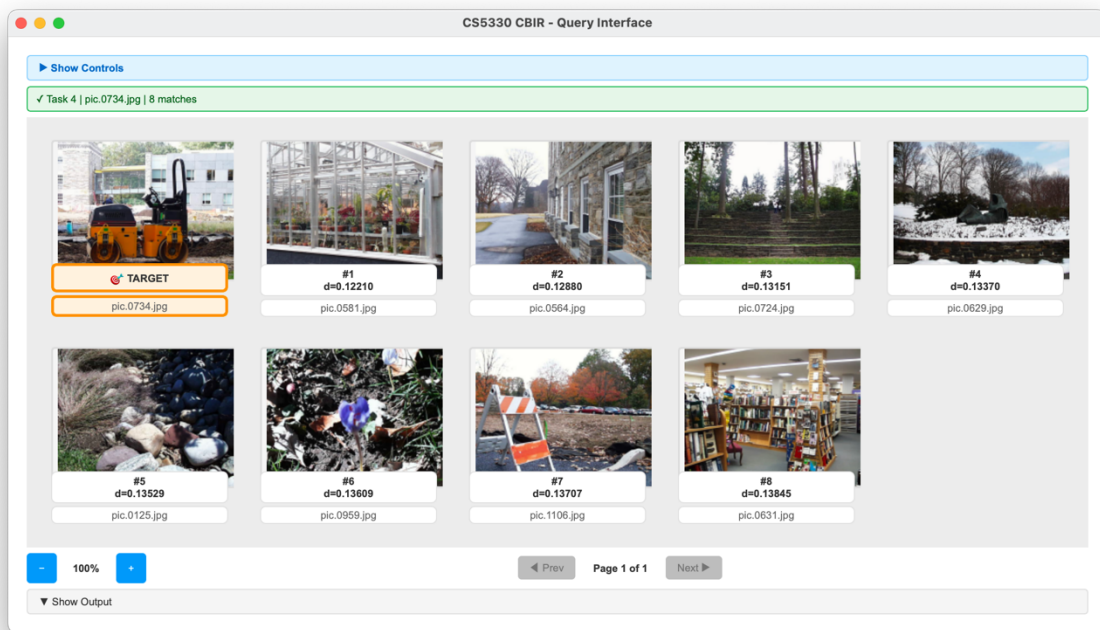
For pic.0948 (mascot), and pic.0734 (construction equipment), it's the same case — DNN outperforms the rest. Mascot and construction equipment are both objects that have a clear outline, and clear semantics, and they occupy a large portion of the image as the main subject. And DNN is exceptionally good at handling these kinds of images.

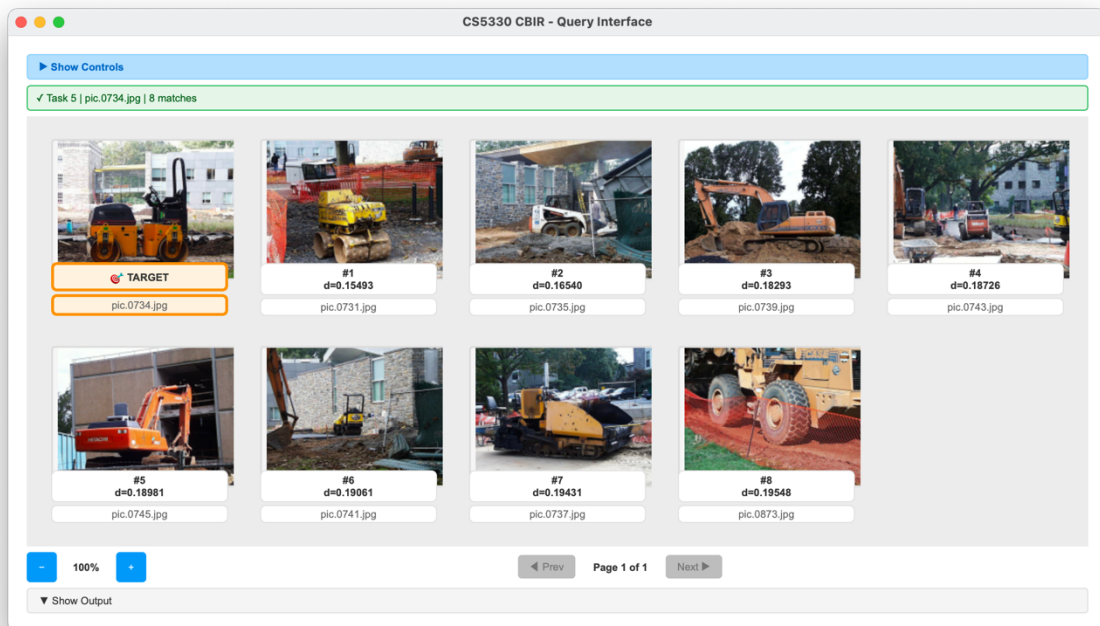
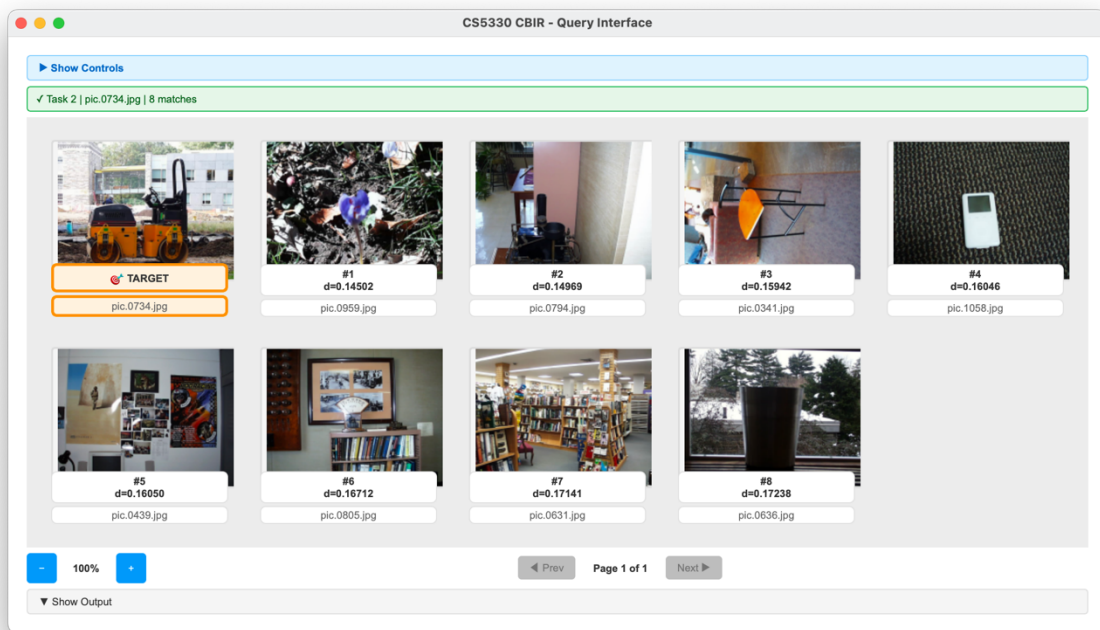
So my conclusion is:

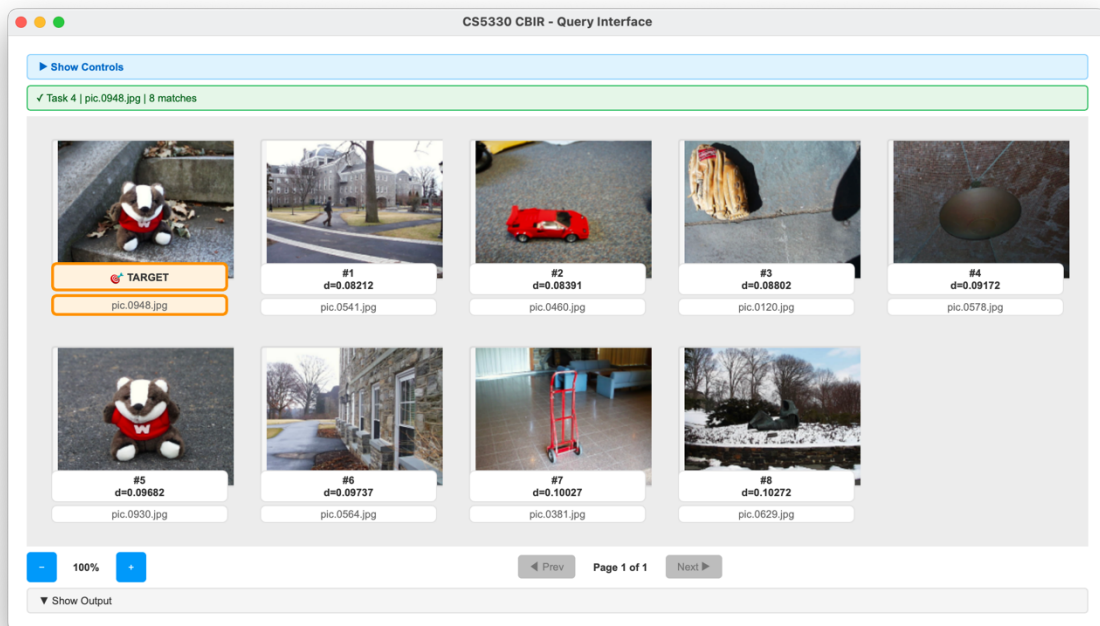
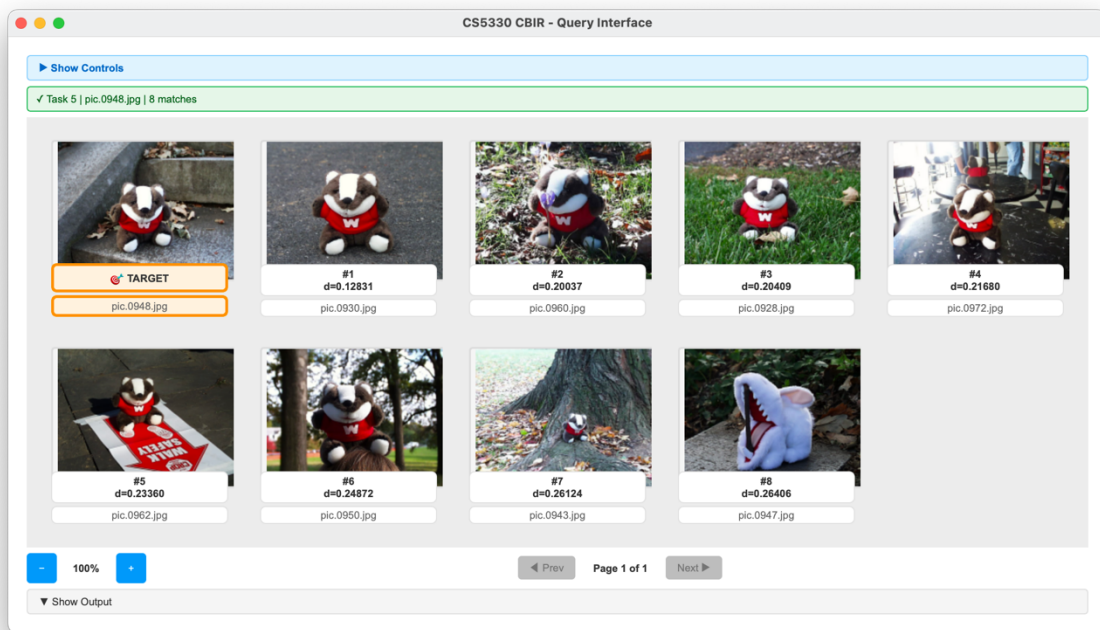
when the target has clear semantics, DNN usually performs well. But when the target lacks clear semantic content, for example, abstract patterns, textures, classical features might perform well.

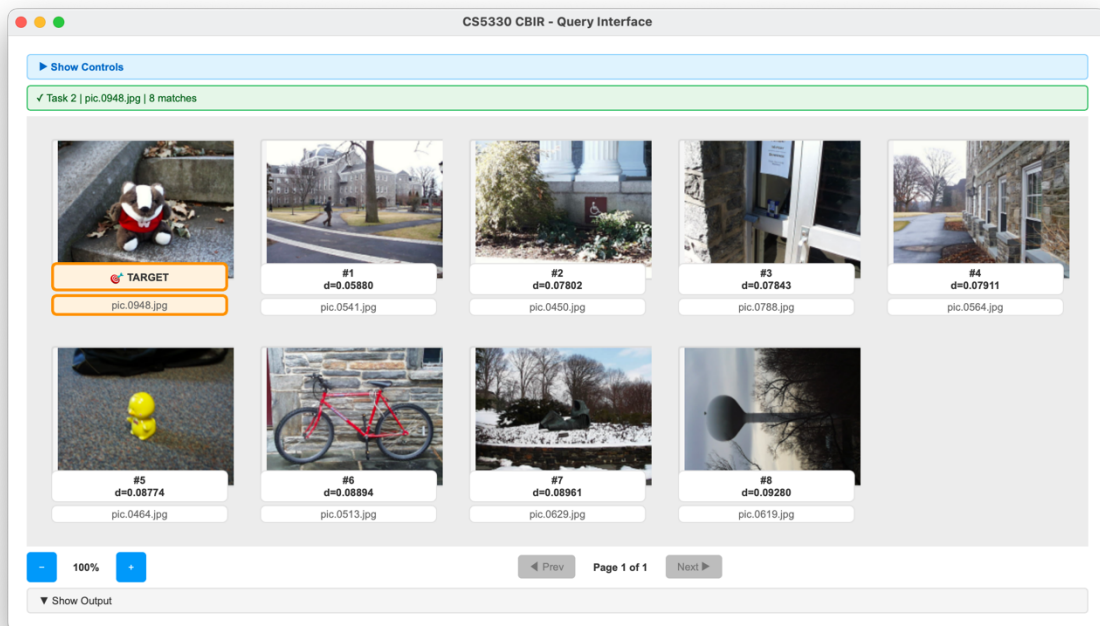
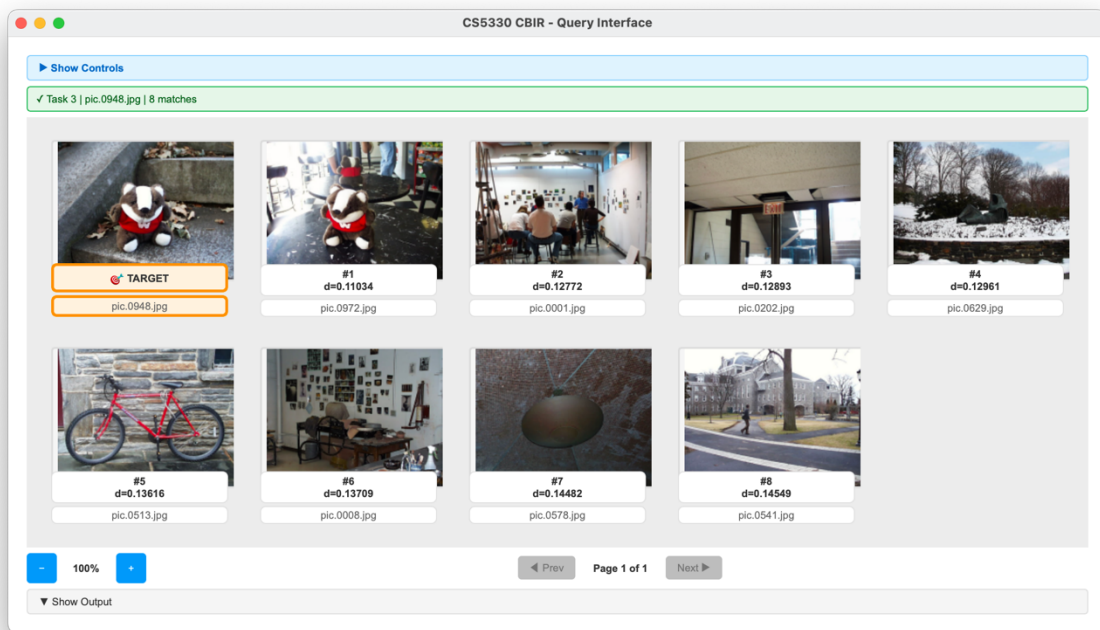
If we only care about the texture or the color, not about the content, then classical features are more suitable.

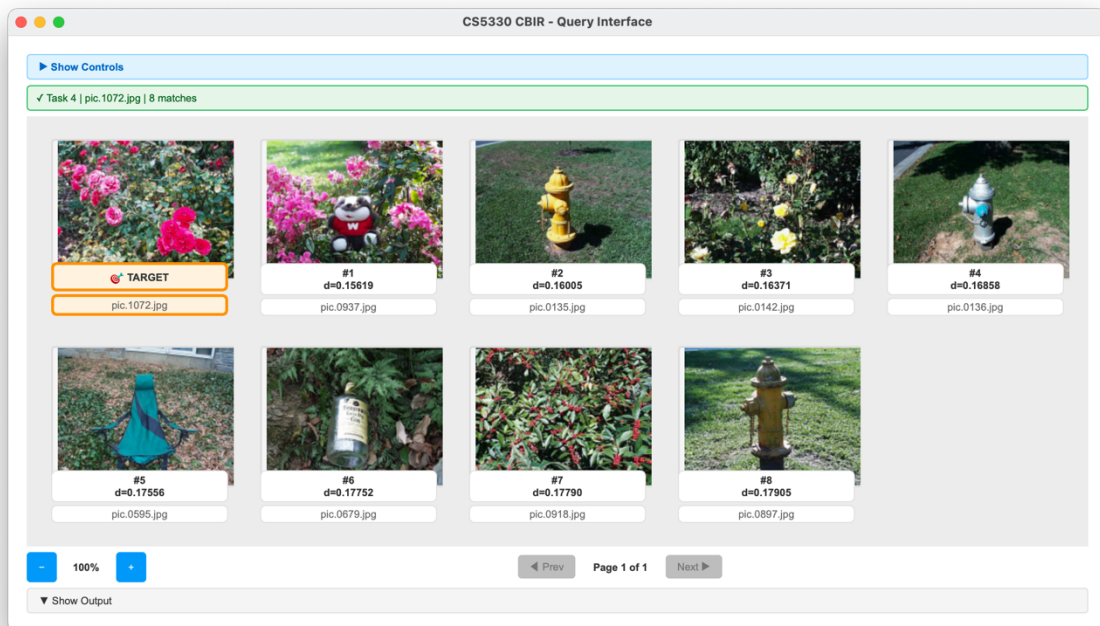
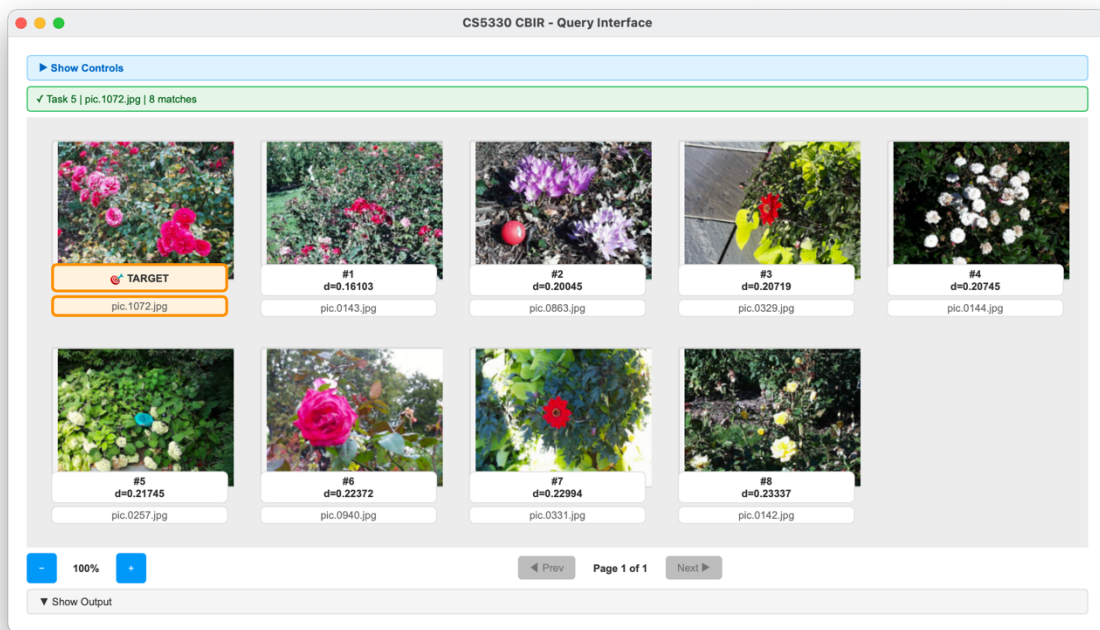
Below are the screenshots of the retrieved results:

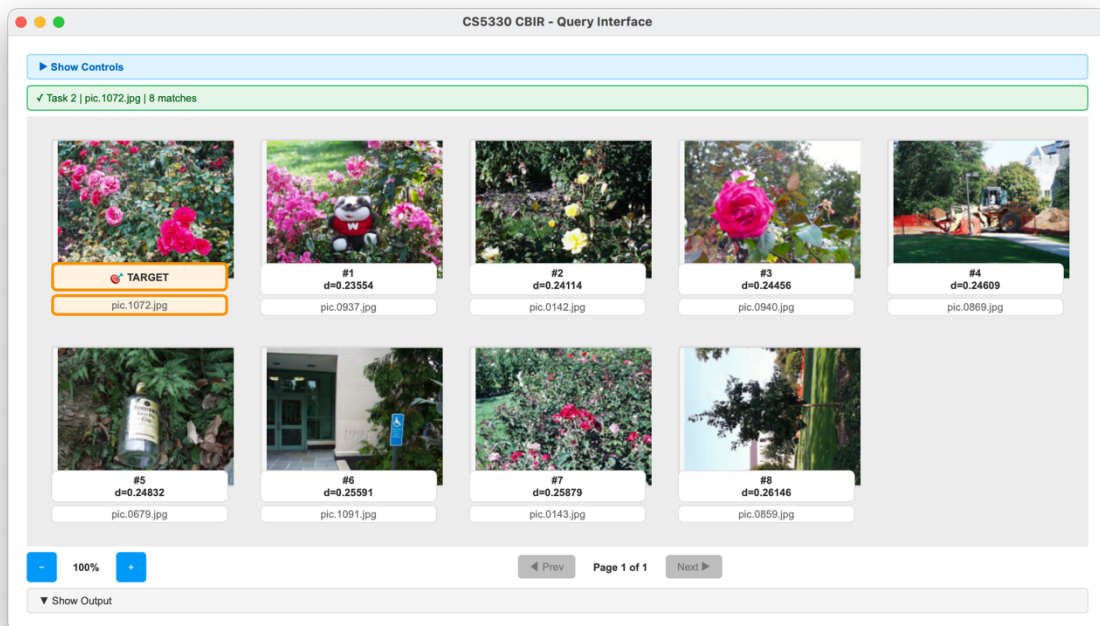
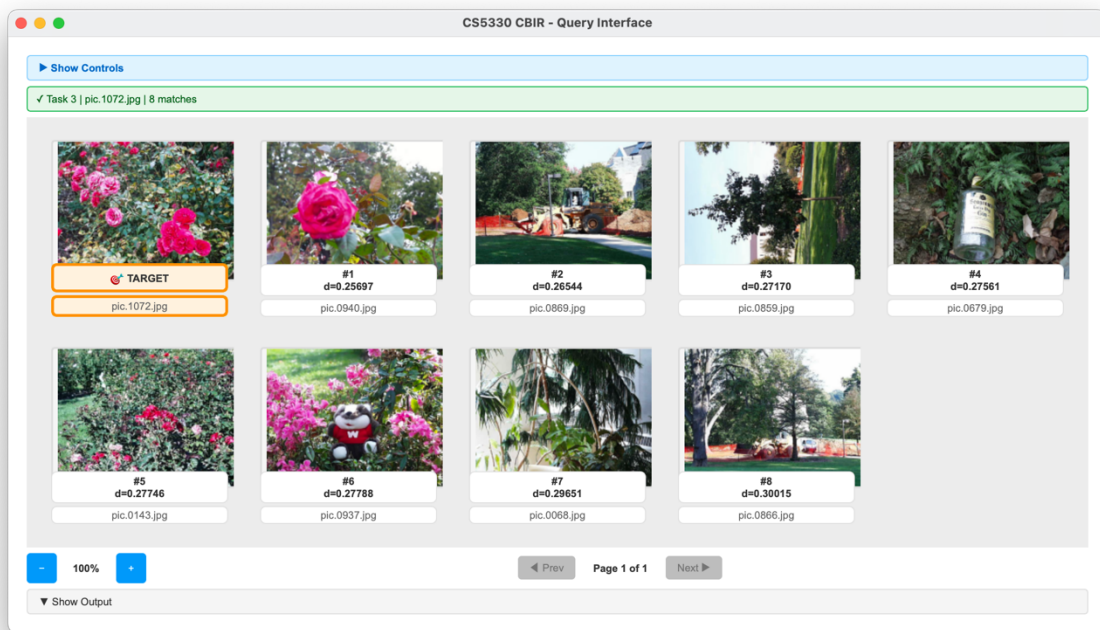












Task 7:

My type of image of interest: grass/lawn

The feature vector I designed:

feat = {green_ratio, avg_h, avg_s, avg_v, (cnt > 0) ? 1.0f : 0.0f};

- [0] green_ratio: 0.65 (65% of image is grass)
- [1] avg_h: 0.45 (Hue ~80° - typical grass green)
- [2] avg_s: 0.60 (Moderately saturated)
- [3] avg_v: 0.55 (Medium brightness)
- [4] has_green: 1.0 (Grass detected)

For distance metric, I combined both the embedding distance and my custom grass feature distance because the former is responsible for finding the similar scenes, or semantic similarity; while the latter is responsible for color-based similarity.

Fusion: 40% DNN + 60% grass features

The grass feature is composed of 5 features with different weights:

Hue (×5): Most important, is it green, not orange? Is it the right shade of green?

Saturation (×3): Important

Green ratio (×2): Important

Value (×1): Less important, brightness varies with lighting

Has-green (×0.5): Least important, a flag

Besides, I also used morphological operations to remove noise and fill gaps.

Results:

I chose these two images as targets:

pic.0408.jpg and pic.0492.jpg

Among the results of both queries:

Top 5: All contain grass/outdoor vegetation

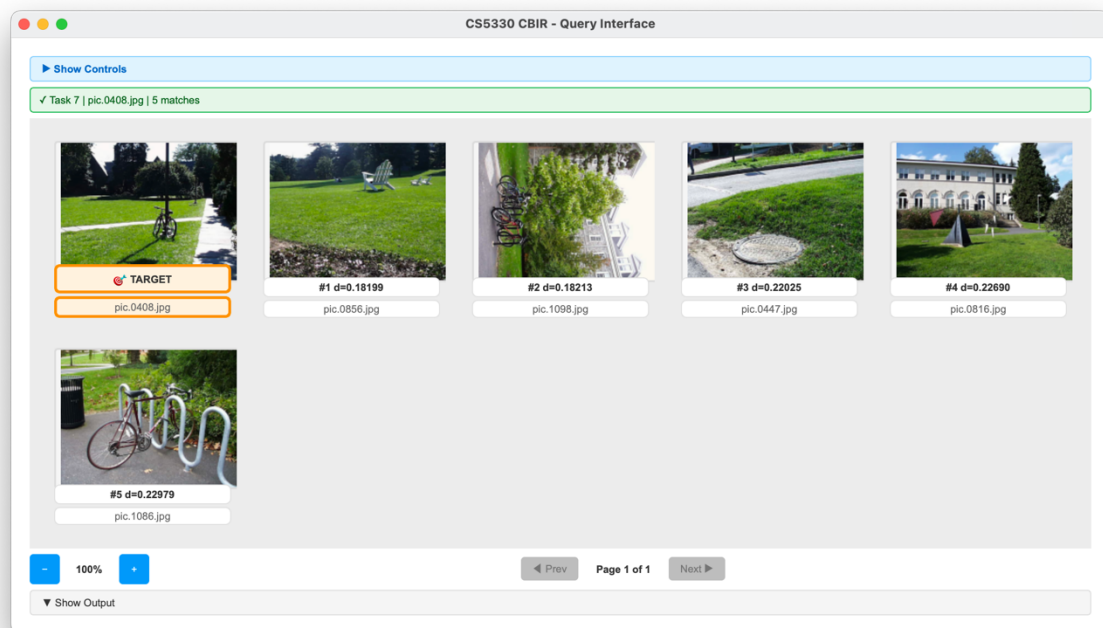
Bottom 5: All indoor/non-grass scenes

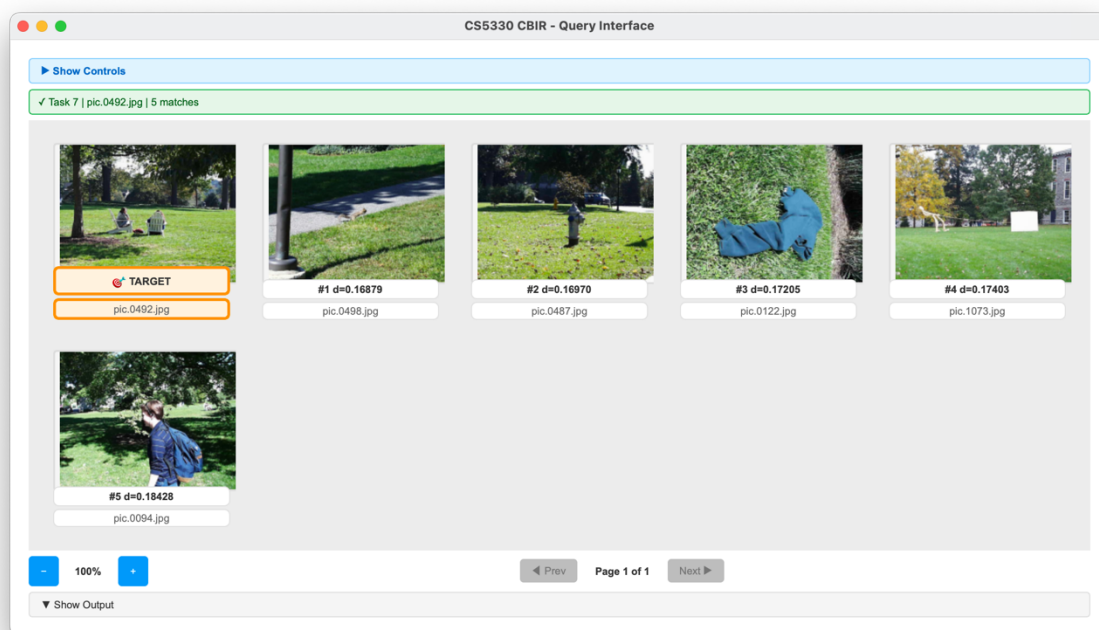
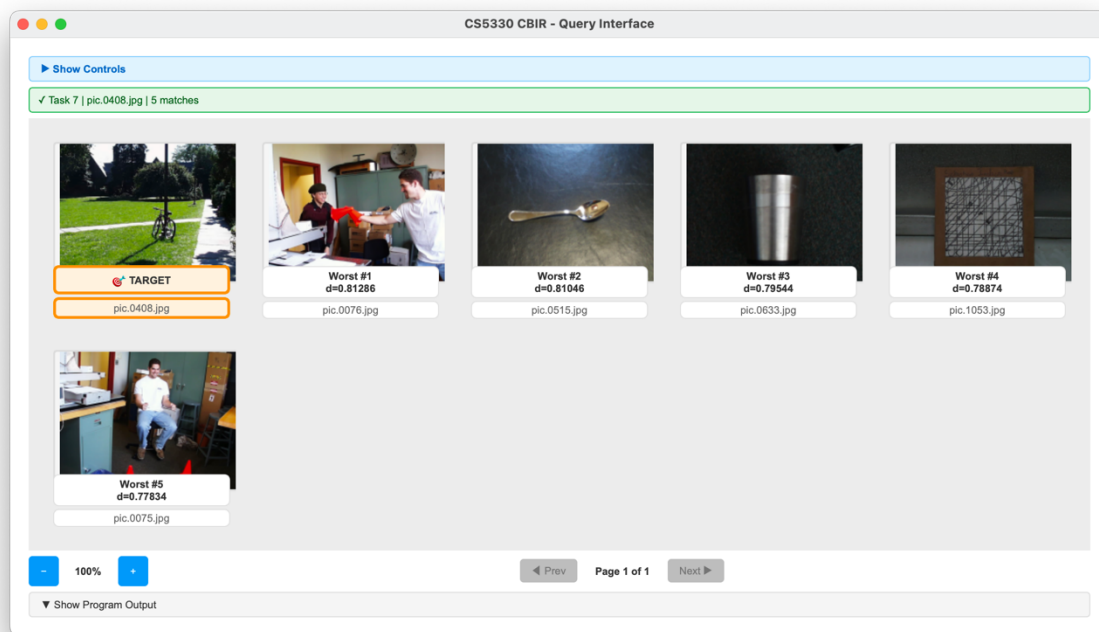
The algorithm successfully distinguishes grass from non-grass images.

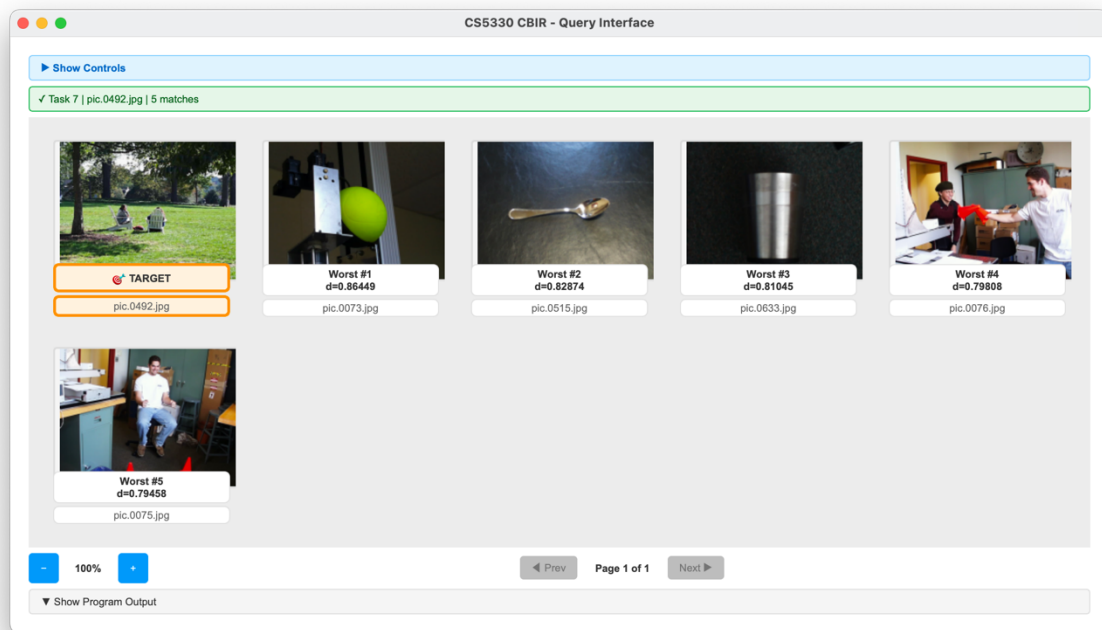
My custom HSV-based grass features provide explicit green color

verification, which complements the ResNet embeddings' semantic understanding.

Below are the screenshots of the retrieved results:



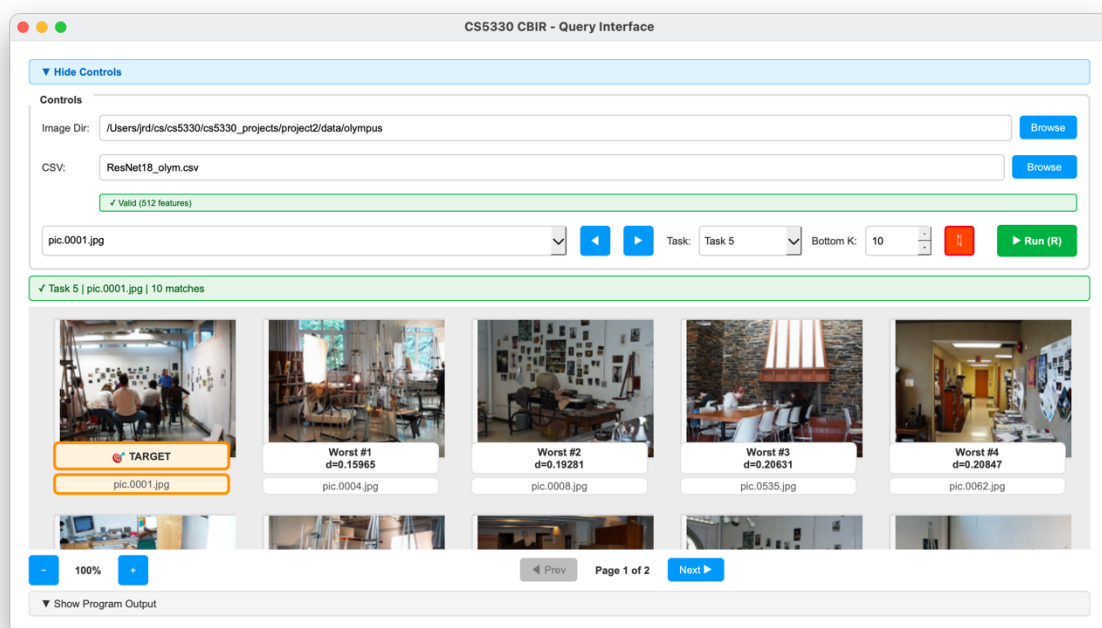




Extension:

I built a PyQt6 GUI for the image retrieval system, which makes it easier to use and test. The interface allows users to select images, choose different feature tasks, and view the ranked matching results visually (either top k or bottom k). It automatically manages CSV files with persistent caching across tasks and sessions, validates feature dimensions, and can auto-build feature csvs for Tasks 1-4.

Example screenshot:



A short reflection of what I learned:

Through this project, I gained a deeper understanding of how we can quantify visual similarity.

And I learnt the difference between classical features and deep network embeddings in a practical way. The former can capture color and texture similarity, while the latter can capture semantic similarity. So, retrieval quality depends not only on features but also on how similarity is measured.

I become better at organizing source files and more familiar with cpp. I also found that building a GUI really supports debugging and evaluation.

Overall, this project strengthened my practical skills in image processing while giving me a clearer perspective on the trade-offs between traditional computer vision techniques and deep learning approaches.

Acknowledgement of any materials or people you consulted for the assignment:

Claude. I consulted it for some implementation details. Also how I can improve my custom feature design in task 7. And I learnt how morphological operations such as MORPH_OPEN and MORPH_CLOSE can help remove small noise regions and fill gaps.

Built on I/O code given by the professor.

I also referred to the OpenCV documentation to understand the behavior of image processing functions.